

CPSC 375 High-Performance Computing

Lecture 22

Derived Types and Barriers

Function for reading user input

```
void Get_input(  
    int  my_rank /* in */,  
    int  comm_sz /* in */,  
    double* a_p  /* out */,  
    double* b_p  /* out */,  
    int* n_p  /* out */) {  
    int dest;  
  
    if (my_rank == 0) {  
        printf("Enter a, b, and n\n");  
        scanf("%lf %lf %d", a_p, b_p, n_p);  
        for (dest = 1; dest < comm_sz; dest++) {  
            MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);  
            MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);  
            MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);  
        }  
    } else { /* my_rank != 0 */  
        MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
        MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
        MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    }  
}
```

Takes too long!

Derived datatypes

- Used to represent any collection of data items in memory by storing both the types of the items and their relative locations in memory.
- The idea is that if a function that sends data knows this information about a collection of data items, it can collect the items from memory before they are sent.
- Similarly, a function that receives data can distribute the items into their correct destinations in memory when they're received.

Derived datatypes

- Formally, consists of a sequence of basic MPI data types together with a displacement for each of the data types.
- Example:

Variable	Address
a	24
b	40
n	48

MPI_Type_create_struct

```
int MPI_Type_create_struct(  
    int                count                /* in */,  
    int                array_of_blocklengths[] /* in */,  
    MPI_Aint           array_of_displacements[] /* in */,  
    MPI_Datatype        array_of_types[]      /* in */,  
    MPI_Datatype*       new_type_p           /* out */);
```

Builds a derived datatype that consists of individual elements that have different basic types.

MPI_Get_address

```
int MPI_Get_address(  
    void* location_p    /* in */,  
    MPI_Aint* address_p /* out */);
```

- Returns the address of the memory location referenced by [location_p](#).
- The special type [MPI_Aint](#) is an integer type that is big enough to store an address on the system.

MPI_Type_commit

```
int MPI_Type_commit (MPI_Datatype* new_mpi_t_p);
```

Allows the MPI implementation to optimize its internal representation of the datatype for use in communication functions.

MPI_Type_free

```
int MPI_Type_free (MPI_Datatype* new_mpi_t_p) ;
```

When we're finished with our new type, this frees any additional storage used.

Get input function with a derived datatype

```
void Build_mpi_type(  
    double* a_p          /* in */,  
    double* b_p          /* in */,  
    int* n_p             /* in */,  
    MPI_Datatype* input_mpi_t_p /* out */) {  
  
    int array_of_blocklengths[3] = {1, 1, 1};  
    MPI_Datatype array_of_types[3] = {MPI_DOUBLE,  
                                      MPI_DOUBLE, MPI_INT};  
  
    MPI_Aint a_addr, b_addr, n_addr;  
    MPI_Aint array_of_displacements[3] = {0};
```

Get input function with a derived datatype

```
MPI_Get_address(a_p, &a_addr);
MPI_Get_address(b_p, &b_addr);
MPI_Get_address(n_p, &n_addr);

array_of_displacements[1] = b_addr - a_addr;
array_of_displacements[2] = n_addr - a_addr;

MPI_Type_create_struct(3, array_of_blocklengths,
                      array_of_displacements, array_of_types,
                      input_mpi_t_p);

MPI_Type_commit(input_mpi_t_p);
} /* Build_mpi_type */
```

Get input function with a derived datatype

```
void Get_input(int my_rank, int comm_sz, double* a_p,
               double* b_p, int* n_p) {
    MPI_Datatype input_mpi_t;

    Build_mpi_type(a_p, b_p, n_p, &input_mpi_t);

    if (my_rank == 0) {
        printf("Enter a, b, and n\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);
    }

    MPI_Bcast(a_p, 1, input_mpi_t, 0, MPI_COMM_WORLD);

    MPI_Type_free(&input_mpi_t);
}
```

Elapsed parallel time

```
double start = MPI_Wtime();  
// ... code to profile ...  
double end = MPI_Wtime();  
printf("Elapsed time: %f seconds\n", end - start);
```

Returns the number of seconds that have elapsed since some time in the past.

MPI_Barrier

```
int MPI_Barrier(MPI_Comm comm);
```

Ensures that no process will return from calling it until every process in the communicator has started calling it.

MPI_Barrier

```
double local_start, local_finish, local_elapsed,
elapsed;
. . .
MPI_Barrier(comm);
local_start = MPI_Wtime();
/* Code to be timed */
. . .
local_finish = MPI_Wtime();
local_elapsed = local_finish - local_start;
MPI_Reduce(&local_elapsed, &elapsed, 1, MPI_DOUBLE,
          MPI_MAX, 0, comm);

if (my_rank == 0)
    printf("Elapsed time = %e seconds\n", elapsed);
```



Assignment 6

- Concurrent Transaction Processing System
 - Accepts concurrent TCP client connections
 - Processes requests using a thread pool
 - Stores all data in DB
 - Uses transactions to ensure atomicity
 - Correctly synchronizes shared memory
 - Shuts down gracefully

The ACID Properties

- **Atomicity:** Every operation within a transaction is bound together as a single atomic unit.
- **Consistency:** A transaction must transition the database from one valid state to another correctly.
- **Isolation:** The system must ensure the final state is identical to the state that would have occurred if they had been run one at a time.
- **Durability:** Once a transaction is finalized (committed), its changes are recorded in non-volatile storage.

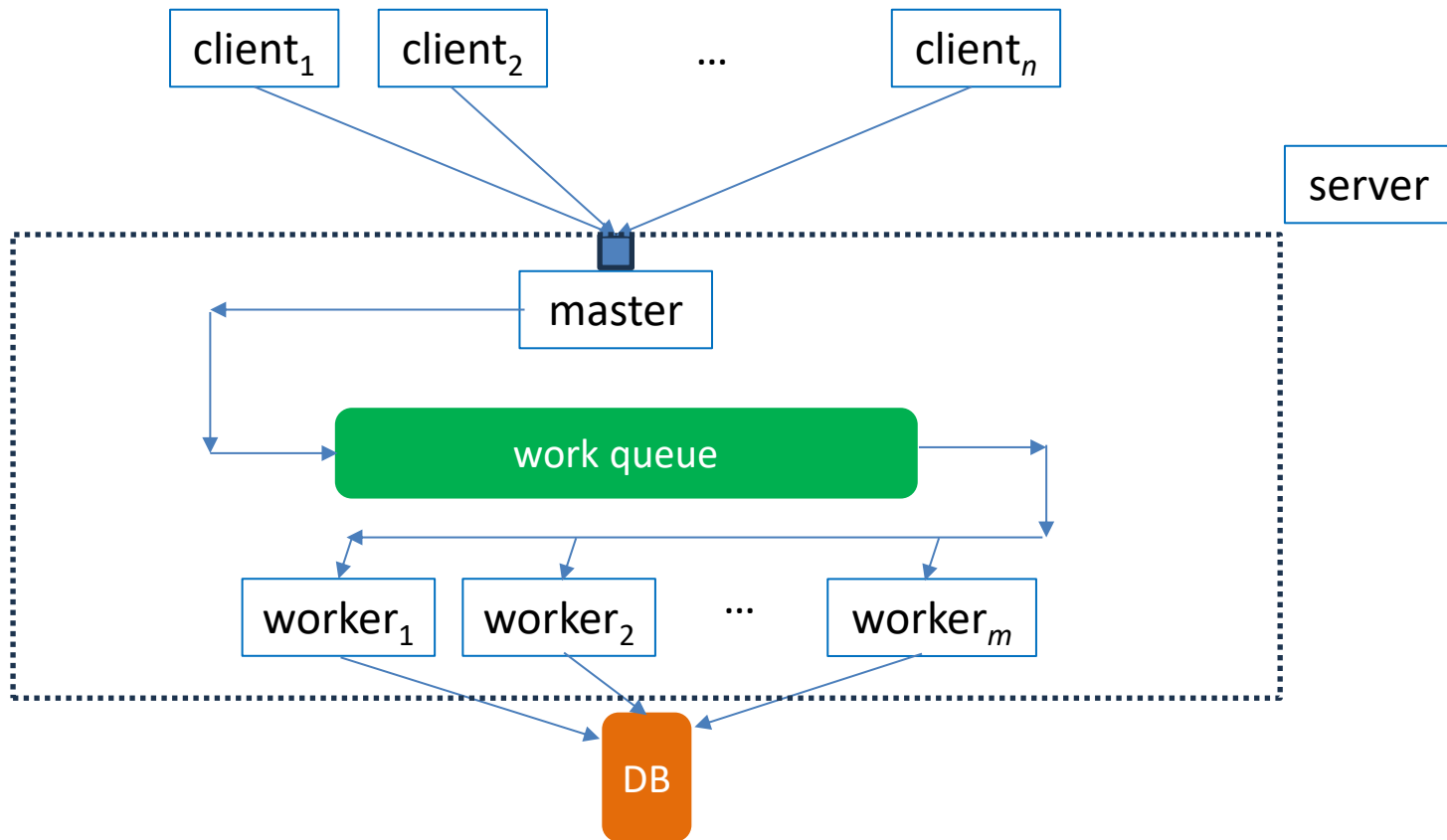
Putting It All Together

- Share memory access
- Signals
- Sockets
- Thread pool
- Critical sections
- Thread synchronization
- Mutexes and condition variables
- Producer/Consumer design

System Architecture

- Server
 - Listens on a TCP port
 - Accepts multiple client connections
 - Pushes client requests into a shared work queue
 - Worker threads process requests
 - Uses DB for persistence
- Client
 - Reads simulation file
 - Sends commands to server

System Workflow



Input Format

T11 CR flights 5 1

fid 1

orig S

dest S

dist 1

price 1

T11 IN flights 5

4821 BOS MIA 1258 200

1934 BOS JFK 215 100

8750 BOS DEN 1971 240

6502 BOS ATL 946 180

7318 BOS ORD 867 170

T11 C

T21 IN flights 2

2945 JFK LAX 2475 280

5012 JFK MIA 1090 190

T21 C

T12 SL flights expen 1

price > 200

T22 UP flights 1

4821 BOS MIA 1258 220

T12 PR expen

T12 C

T22 C

